GREYNOC

CYBERSECURITY OPERATIONS · MICHIGAN

PUBLIC

OPERATOR FIELD REFERENCE

Scripting Fundamentals

Creating, Understanding & Running Scripts

with a Defensive Automation Playbook

```
$ ./greynoc --module scripting --mode defensive
[ok] loading operator reference ...
```

DOCUMENT ID	VERSION	CLASSIFICATION	PUBLISHED
GN-OPS-SCRIPT-001	v1.0	PUBLIC	2026-06-14

Document Control

Field	Value
Document ID	GN-OPS-SCRIPT-001
Title	Scripting Fundamentals for Operators — Creating, Understanding & Running Scripts
Version	v1.0
Classification	PUBLIC — distribution permitted
Published	2026-06-14
Owner	GreyNOC Operations
Template	GN-TPL-PUB-001
Scope	Bash · PowerShell · Python scripting for IT & defensive security

Revision History

Version	Date	Author	Summary
v1.0	2026-06-14	GreyNOC	Initial public release. Expanded source manual with advanced techniques and a defensive automation playbook.

About This Guide

This guide teaches scripting as both a practical skill and a technical discipline. It is written for beginning and intermediate operators across IT, networking, system administration, and defensive security who want to build automation that is **useful, safe, repeatable, and trusted**. The first half covers fundamentals — what scripts are, how they are structured, how they are analyzed, built, and run. The second half goes further than most introductory material: a set of professional techniques that separate fragile scripts from production-grade ones, and a defensive scripting playbook of ready-to-adapt blue-team automation.

NOTE. Every example here is defensive or administrative in nature. The goal is reliable automation and safer operations — not offensive tooling. Adapt all examples to your own environment and test in a lab before touching production.

How to Read the Callouts

Marker	Meaning
TIP	A practice that materially improves quality, safety, or maintainability.
CAUTION	A common pitfall or a place where things quietly go wrong.
DANGER	An action that can cause irreversible damage if mishandled.
NOTE	Context or clarification worth keeping in mind.

Contents

1. What Is a Script?

A script is a set of instructions, written in a scripting language, that tells a computer or system to perform specific tasks. Unlike full software applications, scripts are usually smaller, more focused, and designed to automate work or control existing programs, services, and operating-system functions. A script is essentially a technical recipe: it gives the system step-by-step instructions to complete a job.

Scripts are used to create and manage files, install software, scan networks, collect system information, parse logs, automate backups, start and stop services, test security controls, configure systems, run scheduled jobs, and interact with APIs. In short, wherever a task is repetitive, error-prone by hand, or needs to run unattended, a script earns its place.

TIP. If you can describe a task as a clear sequence of steps, you can usually script it. If you *cannot* describe it clearly, that is a sign to slow down and define the goal before writing code.

2. A Brief History of Scripting

Scripting began as a way to automate command-line work. Early operators needed to run repeated commands without retyping them, which led to shell scripts, batch files, and job-control scripts.

2.1 From Command Automation to Shells

Early operating systems were driven entirely from the command line. As systems grew more complex, users needed to group commands together — and command scripts were born. Unix popularized the approach with its philosophy of small, composable tools, giving rise to shells such as `sh`, `bash`, `ksh`, and `zsh`. Shell scripting remains central to Linux, macOS, cloud, and security workflows today.

2.2 Windows: Batch and PowerShell

DOS and early Windows used batch files (`.bat`, `.cmd`) for login scripts and setup tasks. PowerShell later became the major Windows automation platform, working with structured *objects* rather than plain text — ideal for Active Directory, Microsoft 365, Azure, and endpoint management.

2.3 Python and Modern Scripting

Python rose to prominence because it is readable, powerful, and flexible. Today scripting underpins DevOps, cloud engineering, cybersecurity, data science, testing, and AI workflows. Modern scripting is no longer just about saving time — it is about building repeatable, scalable, controlled processes.

3. Common Scripting Languages

Different languages suit different environments. Pick the one that fits the system you are operating and the task at hand.

Language	Strong fit for	Typical use
Bash	Linux / Unix, servers, DevOps	File ops, service control, cron jobs, glue between CLI tools
PowerShell	Windows, Microsoft ecosystem	AD, M365, Azure, endpoint automation, security reporting
Python	Cross-platform, advanced automation	APIs, log/data parsing, security tooling, reusable utilities
JavaScript	Web and Node.js	Browser behavior, APIs, server-side automation
Perl	Legacy systems, text processing	Log parsing, older automation workflows
Ruby	Web, infra tooling	Configuration management, DevOps tooling

TIP. When a task spans both Windows and Linux, Python is usually the safest cross-platform choice. When you are deep inside one OS, the native shell (PowerShell or Bash) is often simpler and faster.

4. Anatomy of a Script

Syntax differs between languages, but the structural elements are remarkably consistent.

4.1 Comments

Comments explain intent and are ignored by the interpreter. Good comments explain *why* something happens, not merely restate *what* the line does.

```
# Weak: restates the obvious
print(name)

# Strong: explains intent
# Display the current user being processed in the audit report
print(name)
```

4.2 Variables

Variables store text, numbers, file paths, user input, command output, API responses, and configuration values.

```
# Python            # Bash              # PowerShell
username = "admin"  username="admin"     $username = "admin"
```

4.3 Input and Output

Input can come from a user, a file, a command-line argument, a network request, an API, or another program. Output may be screen text, a file, a log entry, a report, a database update, or an alert.

```
# Python: simple input / output
name = input("Enter your name: ")
print("Hello", name)
```

4.4 Conditional Logic

Conditionals let a script make decisions.

```
# Python
status = "running"
if status == "running":
    print("Service is active")
else:
    print("Service is inactive")
```

4.5 Loops

Loops repeat actions — processing files, checking multiple users, scanning multiple hosts, or reading log lines.

```
# Python                  # Bash
for n in range(1, 6):      for n in 1 2 3 4 5; do
    print(n)                   echo "$n"
                           done
```

4.6 Functions

Functions group reusable logic, making scripts cleaner, testable, and easier to troubleshoot.

```
def greet_user(name):
    print("Hello", name)

greet_user("operator")
```

4.7 Error Handling

Good scripts expect failure and handle it safely rather than crashing or — worse — continuing in a broken state.

```
# Python
try:
    with open("report.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("The file was not found.")
```

4.8 Exit Codes

Exit codes tell the operating system whether a script succeeded. They are essential for automation, scheduling, monitoring, and CI/CD. By convention, `0` means success and non-zero means failure.

Code	Conventional meaning
`0`	Success
`1`	General / catch-all error
`2`	Misuse — bad arguments or usage
`3–125`	Application-specific failures you define
`126`	Command found but not executable
`127`	Command not found
`130`	Terminated by Ctrl+C (SIGINT)

5. Technical Analysis of Scripts

Technical analysis means reviewing how a script works, what it affects, and whether it is safe, efficient, and reliable — *before* you trust it on a real system. Run through these lenses for any script you write, inherit, or download.

5.1 Purpose

State plainly what the script does. A clear purpose makes a script easier to test and safer to run.

Weak purpose	Strong purpose
“This script checks stuff.”	“Collect failed-login attempts from the Linux auth log and write a summary report.”

5.2 Environment & Dependencies

A script may work in one environment and fail in another. Confirm the OS, interpreter, required software, permissions, file paths, network access, installed modules, and environment variables. List dependencies explicitly — which modules are imported, which external commands are called, whether API keys are required, and whether the script can run offline.

5.3 Input Analysis

Unsafe input can break a script or create a security hole. Check where input comes from, whether it is trusted and validated, and whether special characters are handled. The classic danger is building a shell command from untrusted input.

```
# UNSAFE — invites command injection
import os
target = input("Enter host: ")
os.system("ping " + target)        # "8.8.8.8; rm -rf ~" would execute

# SAFER — pass arguments as a list, no shell string parsing
import subprocess
target = input("Enter host: ")
subprocess.run(["ping", "-c", "4", target])
```

DANGER. Never concatenate untrusted input into a command string. Pass arguments as a list and avoid `shell=True`. This single habit prevents an entire class of injection vulnerabilities.

5.4 Logic, Output & Edge Cases

Verify the script makes correct decisions and handles the unexpected: what happens if no files are found, the network is down, input is malformed, a command fails, or the script is interrupted mid-run? Output should be clear and controlled — does it overwrite files, write logs, or expose sensitive data?

Poor output	Good output
done	[INFO] Backup completed: /backup/server1-2026-06-14.tar.gz (412 MB)

5.5 Security, Reliability & Maintainability

Security review asks whether the script needs elevated rights, handles secrets, downloads or executes remote code, or modifies users, permissions, or firewall rules. Reliability asks whether it validates input, logs key events, cleans up temporary files, and can be run more than once safely — an **idempotent** script. Maintainability asks whether names are clear, structure is simple, and configuration is separated from logic.

```
# Idempotent: safe to run repeatedly
import os
os.makedirs("reports", exist_ok=True)   # no error if it already exists
```

6. Building Scripts Properly

Creating a script is a process, not just typing code. The following sequence keeps scripts clean and predictable.

- **Define the goal** — what it does, what system it touches, its inputs and outputs, and what success looks like.
- **Choose the language** to match the environment and task.
- **Plan** the steps in plain language before writing code.
- **Build a minimum working version** that does only the core task.
- **Add logic**, then **error handling**, then **documentation**.
- **Test** under normal and abnormal conditions, then **review** before production.

6.1 Always Start With a Header

```
"""
GreyNOC Script
Name:         disk_check.py
Purpose:      Report a warning when disk usage exceeds a threshold.
Author:       GreyNOC
Version:      1.0
Date:         2026-06-14
Requirements: Python 3
Usage:        python disk_check.py
"""
```

6.2 Name Clearly and Separate Configuration

```
# Weak
x = 80
y = "/var/log/auth.log"

# Strong — names explain themselves, config sits up top and is easy to change
FAILED_LOGIN_LIMIT = 10
AUTH_LOG_PATH      = "/var/log/auth.log"
REPORT_FILE        = "failed_login_report.txt"
```

6.3 Validate Input and Log Key Events

```
import logging
logging.basicConfig(filename="script.log", level=logging.INFO)

age = input("Enter age: ")
if not age.isdigit():
    logging.warning("Rejected non-numeric age input")
    print("Age must be a number.")
else:
    logging.info("Accepted age input")
    print(f"Age entered: {age}")
```

6.4 Never Hardcode Secrets

```
# Poor — secret committed into source forever
password = "MyPassword123"

# Better — read from the environment at run time
import os
password = os.getenv("APP_PASSWORD")
```

CAUTION. A secret committed to version control is compromised even after you delete it — it stays in history. Rotate any credential that has ever been committed, and keep secrets out of code from the start.

6.5 Use Version Control

Store scripts in Git so you can track what changed, who changed it, when, and why — and roll back when needed.

```
git status
git add disk_check.py
git commit -m "Add disk usage check with threshold alerting"
```

7. Running Scripts Safely

Scripts can run manually, on a schedule, in response to an event, or as a stage in a pipeline. However they run, the safety discipline is the same.

7.1 Running by Language

Language	Run command	Notes
`Python`	python3 script.py	On some systems python points to Python 2; prefer python3
`Bash`	bash script.sh or ./script.sh	Needs chmod +x and a #!/bin/bash shebang to run directly
`PowerShell`	.\script.ps1	Execution policy may block scripts; scope changes narrowly
`Batch`	script.bat	Run from Command Prompt

7.2 Scheduling

```
# Linux cron — run a backup every day at 02:00
0 2 * * * /usr/bin/python3 /home/operator/backup.py
```

On Windows, Task Scheduler runs PowerShell, batch, or Python scripts on a schedule or in response to events such as logon, startup, or a triggered alert.

7.3 Passing Arguments

```
# Python
import sys
print(sys.argv)
```

```
# Bash
   #!/bin/bash
   echo "Host: $1"
```

```
# PowerShell
   param([string]$Computer)
  Write-Output "Host: $Computer"
```

7.4 The Safe-Execution Checklist

- Read the entire script and confirm its source before running it.
- Identify anything that deletes, overwrites, formats, or disables.
- Check the permissions it needs — run with least privilege.
- Test in a lab or with a dry run first; back up important data.
- Watch the output as it runs and review the logs afterward.

DANGER. Never pipe a remote script straight into a shell — `curl http://site/x.sh | bash` runs unseen code instantly. Download, read it, verify the source, *then* run it.

7.5 High-Risk Commands to Review Carefully

Command	Why it is dangerous
`rm -rf <path>`	Recursive, forced deletion — no undo
`Remove-Item -Recurse -Force`	PowerShell equivalent of irreversible bulk deletion
`dd / mkfs`	Can overwrite or format entire disks
`chmod -R / chown -R`	Sweeping permission changes can break a system
`:(){ :|:& };:`	A fork bomb — exhausts system resources

8. Advanced Techniques & Pro Tips

These are the habits that separate a script that merely works from one you can trust unattended in production. Most cost only a few extra lines.

8.1 Bash Hardening

Start every serious Bash script with strict mode, and clean up after yourself with a trap.

```
#!/bin/bash
set -euo pipefail          # exit on error, unset var, or failed pipe
IFS=$'\n\t'                # safer word-splitting

WORKDIR="$(mktemp -d)"     # private temp dir
cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT          # always runs, even on error or Ctrl+C

# Always quote variables to survive spaces and special characters
echo "Working in: $WORKDIR"
```

- **set -e** stops on the first error instead of barreling ahead in a broken state.
- **set -u** turns a typo'd or unset variable into an error rather than silent emptiness.
- **set -o pipefail** makes a pipeline fail if *any* stage fails, not just the last.
- **trap cleanup EXIT** guarantees temp files are removed however the script ends.
- Quote every variable (`"$var"`) and use `[[ ]]` over `[ ]` for tests.

TIP. Run every shell script through **shellcheck** before committing. It catches quoting bugs, unsafe patterns, and portability issues that are easy to miss by eye.

8.2 Python: Production Patterns

Reach for the standard library's tools rather than hand-rolling fragile equivalents.

```
#!/usr/bin/env python3
"""Disk usage check — production-shaped example."""
import argparse, logging, sys
from pathlib import Path

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger("disk_check")

def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Warn when disk usage is high.")
    p.add_argument("--path", type=Path, default=Path("/"))
    p.add_argument("--threshold", type=int, default=80)
    p.add_argument("--dry-run", action="store_true")
    return p.parse_args()

def main() -> int:
    args = parse_args()
    import shutil
    total, used, _ = shutil.disk_usage(args.path)
    pct = used / total * 100
    log.info("Usage on %s: %.1f%%", args.path, pct)
    if pct >= args.threshold:
        log.warning("Disk usage above threshold (%d%%)", args.threshold)
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

- Use **argparse** instead of reading `sys.argv` by hand — you get help text, types, and validation for free.
- Use the **logging** module instead of `print` — levels, timestamps, and routing to files.
- Use **pathlib.Path** for file paths so code works across operating systems.
- Add **type hints** and a **--dry-run** flag so the script can show its intent without acting.
- Pin dependencies in `requirements.txt` and work inside a virtual environment.

8.3 PowerShell: Production Patterns

PowerShell has first-class support for safe, self-documenting scripts.

```
<#
.SYNOPSIS  Restart a service safely with -WhatIf support.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory)][ValidateNotNullOrEmpty()]
    [string]$ServiceName
)
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

try {
    if ($PSCmdlet.ShouldProcess($ServiceName, "Restart service")) {
        Restart-Service -Name $ServiceName
        Write-Output "Restarted $ServiceName"
    }
}
catch {
    Write-Error "Failed: $_"
    exit 1
}
```

- **[CmdletBinding(SupportsShouldProcess)]** gives you `-WhatIf` and `-Confirm` automatically.
- **Set-StrictMode** and `$ErrorActionPreference='Stop'` turn silent failures into real errors.
- Validate parameters with attributes like `[ValidateNotNullOrEmpty()]`.
- Use approved verbs (Get, Set, New, Restart) and comment-based help for discoverability.
- Avoid `Invoke-Expression` on any untrusted string.

8.4 Cross-Cutting Habits

Habit	Why it matters
Idempotency	A re-run causes no harm — critical for scheduled jobs and recovery.
Dry-run / --check mode	Show what would change before changing anything.
Structured logging (JSON)	Machine-readable logs feed straight into SIEM and dashboards.
Meaningful exit codes	Let schedulers and pipelines react correctly to outcomes.
Secrets via env / vault	Keeps credentials out of source and history.
Pinned dependencies	Reproducible runs; protects against supply-chain drift.
Linting in CI	shellcheck / ruff / PSScriptAnalyzer catch issues before merge.

TIP. A secrets hierarchy, best to worst: a managed secret store or vault → a managed identity → environment variables → a permission-restricted config file. Hardcoded secrets are never acceptable.

9. Defensive Scripting Playbook

Automation is a force multiplier for defenders. The scripts below are **read-and-alert** by design — they observe, baseline, and report rather than take destructive action — which makes them safe to deploy early and to run on a schedule. Treat them as starting points: adapt paths, thresholds, and alerting to your environment, and test in a lab first.

9.1 The Defensive Automation Mindset

- **Baseline, then detect drift.** Most detection is comparing now against a known-good snapshot.
- **Prefer observation over action.** Alert a human before automating a destructive response.
- **Make output SIEM-friendly.** Structured, timestamped lines integrate with existing tooling.
- **Fail safe and loud.** A monitoring script that dies silently is worse than none at all.

9.2 Idea Catalog

A menu of high-value defensive scripts, mapped to common security functions. Several are built out in full below.

Defensive script	Function	What it does
Failed-login monitor	Detect	Counts auth failures per source IP and flags brute-force attempts
File integrity monitor	Detect	Hashes critical files and reports any change, addition, or removal
TLS expiry checker	Protect	Warns before certificates expire and cause outages
Listening-port baseline	Detect	Diffs open ports against an approved allowlist
Verified backup	Recover	Creates backups and proves their integrity with checksums
Resource health monitor	Detect	Tracks disk, memory, and CPU against thresholds
New-account auditor	Detect	Flags newly created or newly privileged local accounts
SUID / sudo auditor	Identify	Inventories privilege-granting binaries and rules
Cron / task auditor	Detect	Baselines scheduled jobs and reports unexpected changes
Patch-status checker	Identify	Reports pending security updates per host
IOC enrichment	Respond	Looks an IP or hash up against threat-intel sources
Secret scanner	Protect	Scans repos for accidentally committed keys and tokens
Config drift detector	Protect	Compares live configuration to an approved baseline
Web availability probe	Detect	Checks reachability, status code, and cert health together

9.3 Worked Example — Failed-Login Monitor (Bash)

Parses the authentication log, counts failed attempts per source IP, and reports any that cross a threshold. Read-only: it alerts rather than blocks, so it is safe to schedule immediately.

```
#!/bin/bash
# GreyNOC — failed_login_monitor.sh
# Purpose: report source IPs exceeding a failed-login threshold.
set -euo pipefail

AUTH_LOG="${1:-/var/log/auth.log}"
THRESHOLD="${2:-10}"

[[ -r "$AUTH_LOG" ]] || { echo "ERROR: cannot read $AUTH_LOG" >&2; exit 1; }

echo "[INFO] Scanning $AUTH_LOG (threshold: $THRESHOLD)"
grep -a "Failed password" "$AUTH_LOG" \
  | grep -aoE "from [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" \
  | awk '{print $2}' | sort | uniq -c | sort -rn \
  | while read -r count ip; do
        if (( count >= THRESHOLD )); then
            echo "[ALERT] $ip — $count failed attempts"
        fi
    done
echo "[INFO] Scan complete"
```

NOTE. For automated blocking, pair this with a maintained tool such as fail2ban rather than scripting bans by hand — the edge cases (allowlists, lockout windows) are easy to get wrong.

9.4 Worked Example — File Integrity Monitor (Python)

Builds a SHA-256 baseline of files you care about, then on later runs reports anything added, changed, or removed. Idempotent and safe to run on a schedule.

```
#!/usr/bin/env python3
"""GreyNOC — fim.py : baseline and detect changes to watched files."""
import argparse, hashlib, json, sys
from pathlib import Path

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()

def snapshot(paths):
    out = {}
    for base in paths:
        for p in Path(base).rglob("*"):
            if p.is_file():
                out[str(p)] = sha256(p)
    return out

def main() -> int:
    ap = argparse.ArgumentParser(description="File integrity monitor.")
    ap.add_argument("paths", nargs="+")
    ap.add_argument("--baseline", default="fim_baseline.json")
    ap.add_argument("--init", action="store_true", help="write a new baseline")
    args = ap.parse_args()

    current = snapshot(args.paths)
    bfile = Path(args.baseline)

    if args.init or not bfile.exists():
        bfile.write_text(json.dumps(current, indent=2))
        print(f"[INFO] Baseline written: {bfile} ({len(current)} files)")
        return 0

    old = json.loads(bfile.read_text())
    added   = current.keys() - old.keys()
    removed = old.keys() - current.keys()
    changed = {p for p in current.keys() & old.keys() if current[p] != old[p]}

    for p in sorted(added):   print(f"[ALERT] ADDED   {p}")
    for p in sorted(removed): print(f"[ALERT] REMOVED {p}")
    for p in sorted(changed): print(f"[ALERT] CHANGED {p}")
    if not (added or removed or changed):
        print("[OK] No changes detected")
        return 0
    return 1

if __name__ == "__main__":
    sys.exit(main())
```

9.5 Worked Example — TLS Certificate Expiry Checker (Python)

Connects to a host, reads its certificate, and warns when expiry is near — turning a silent future outage into an early, actionable alert.

```
#!/usr/bin/env python3
"""GreyNOC — cert_expiry.py : warn before a TLS certificate expires."""
import argparse, socket, ssl, sys
from datetime import datetime, timezone

def days_left(host: str, port: int = 443) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=8) as sock:
        with ctx.wrap_socket(sock, server_hostname=host) as ssock:
            cert = ssock.getpeercert()
    expires = datetime.strptime(cert["notAfter"], "%b %d %H:%M:%S %Y %Z")
    expires = expires.replace(tzinfo=timezone.utc)
    return (expires - datetime.now(timezone.utc)).days

def main() -> int:
    ap = argparse.ArgumentParser(description="TLS certificate expiry checker.")
    ap.add_argument("host")
    ap.add_argument("--warn", type=int, default=21, help="warn within N days")
    args = ap.parse_args()
    try:
        left = days_left(args.host)
    except Exception as e:
        print(f"[ERROR] {args.host}: {e}")
        return 2
    if left < 0:
        print(f"[ALERT] {args.host}: certificate EXPIRED")
        return 1
    if left <= args.warn:
        print(f"[WARN] {args.host}: expires in {left} days")
        return 1
    print(f"[OK] {args.host}: {left} days remaining")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

9.6 Worked Example — Listening-Port Baseline & Drift (Bash)

Compares currently listening ports against an approved allowlist and flags anything unexpected — a cheap, effective way to catch a new service or an unwanted listener.

```
#!/bin/bash
# GreyNOC — port_drift.sh : flag listening ports not on the allowlist.
set -euo pipefail
ALLOWLIST="${1:-allowed_ports.txt}"   # one port per line, e.g. 22

[[ -r "$ALLOWLIST" ]] || { echo "ERROR: missing $ALLOWLIST" >&2; exit 1; }

current="$(ss -ltn | awk 'NR>1 {n=split($4,a,":"); print a[n]}' | sort -un)"

echo "[INFO] Auditing listening TCP ports"
while read -r port; do
    [[ -z "$port" ]] && continue
    if ! grep -qx "$port" "$ALLOWLIST"; then
        echo "[ALERT] Unexpected listening port: $port"
    fi
done <<< "$current"
echo "[INFO] Audit complete"
```

9.7 Worked Example — Verified Backup (Bash)

Creating a backup is only half the job; proving you can trust it is the other half. This archives a directory, records a checksum, and verifies the archive is intact.

```
#!/bin/bash
# GreyNOC — verified_backup.sh
set -euo pipefail
SRC="${1:?usage: verified_backup.sh <source-dir> <dest-dir>}"
DEST="${2:?usage: verified_backup.sh <source-dir> <dest-dir>}"

stamp="$(date +%Y-%m-%d_%H%M%S)"
archive="$DEST/backup_${stamp}.tar.gz"
mkdir -p "$DEST"

echo "[INFO] Archiving $SRC -> $archive"
tar -czf "$archive" -C "$(dirname "$SRC")" "$(basename "$SRC")"

sha256sum "$archive" > "$archive.sha256"
echo "[INFO] Verifying integrity"
sha256sum -c "$archive.sha256"

size="$(du -h "$archive" | cut -f1)"
echo "[INFO] Backup verified: $archive ($size)"
```

TIP. A backup you have never restored is a hope, not a backup. Periodically test a restore into a scratch location — the verified-checksum step above is necessary but not sufficient on its own.

9.8 Worked Example — Resource Health Monitor (Python)

A multi-metric health check that extends the simple disk example into something schedulable, with thresholds and a meaningful exit code for your monitoring system.

```
#!/usr/bin/env python3
"""GreyNOC — health_monitor.py : disk + memory check with exit codes."""
import shutil, sys
from pathlib import Path

DISK_WARN = 85   # percent
MEM_WARN  = 90   # percent

def disk_pct(path="/") -> float:
    total, used, _ = shutil.disk_usage(path)
    return used / total * 100

def mem_pct() -> float:
    # Linux: parse /proc/meminfo (avoids extra dependencies)
    info = {}
    for line in Path("/proc/meminfo").read_text().splitlines():
        k, v, *_ = line.replace(":", "").split()
        info[k] = int(v)
    total, avail = info["MemTotal"], info["MemAvailable"]
    return (total - avail) / total * 100

def main() -> int:
    rc = 0
    d = disk_pct()
    print(f"[{'WARN' if d >= DISK_WARN else 'OK'}] Disk usage: {d:.1f}%")
    if d >= DISK_WARN: rc = 1
    try:
        m = mem_pct()
        print(f"[{'WARN' if m >= MEM_WARN else 'OK'}] Memory usage: {m:.1f}%")
        if m >= MEM_WARN: rc = 1
    except FileNotFoundError:
        print("[INFO] Memory check skipped (no /proc/meminfo)")
    return rc

if __name__ == "__main__":
    sys.exit(main())
```

10. Troubleshooting

When a script fails, work methodically rather than guessing.

- **Read the error message** — it usually names the problem: file not found, permission denied, command not found, module missing, syntax error, timeout.
- **Go to the line number** the interpreter reports first.
- **Add debug output** to print key values while testing.
- **Run one part at a time** — isolate input, logic, output, and error handling.
- **Check permissions and paths** — these differ between systems and are the most common culprits.

```
# Quick debug prints
print(f"DEBUG: file_path={file_path}")     # Python
echo "DEBUG: file_path=$file_path"          # Bash
Write-Output "DEBUG: filePath=$filePath"    # PowerShell
```

10.1 Common Mistakes

- Running unknown scripts without reviewing them.
- Hardcoding passwords, keys, or tokens.
- Ignoring errors and assuming paths or dependencies exist.
- Unclear variable names and no documentation of usage.
- Running as root or administrator when it is not required.
- Overwriting files accidentally and skipping version control.

---

11. Standard Script Templates

Start new scripts from these GreyNOC templates so structure, headers, and error handling are consistent from the first line.

11.1 Python

```
"""
GreyNOC Script
Name:        Purpose:        Author:
Version:     Date:           Requirements:    Usage:
"""
import sys

def main() -> int:
    try:
        print("Script started")
        # --- logic here ---
        print("Script completed successfully")
        return 0
    except Exception as error:
        print(f"ERROR: {error}")
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

11.2 Bash

```
#!/bin/bash
# GreyNOC Script
# Name:  Purpose:  Author:  Version:  Date:  Requirements:  Usage:
set -euo pipefail

echo "Script started"
# --- logic here ---
echo "Script completed successfully"
exit 0
```

11.3 PowerShell

```
<#
GreyNOC Script
Name:  Purpose:  Author:  Version:  Date:  Requirements:  Usage:
#>
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"
try {
    Write-Output "Script started"
    # --- logic here ---
    Write-Output "Script completed successfully"
    exit 0
}
catch {
    Write-Error "ERROR: $_"
    exit 1
}
```

12. Operator Checklists

12.1 Planning

- Purpose is clear.
- Target system is known.
- Language is chosen for the environment.
- Inputs and outputs are defined.
- Required permissions and risks are understood.

12.2 Code

- Header present.
- Clear names; functions for repeated logic.
- Inputs validated; errors handled.
- Output is clear; key events logged.
- No hardcoded secrets.
- Dangerous commands reviewed.

- Meaningful exit codes.

12.3 Testing

- Tested in a safe environment.
- Valid and invalid input tested.
- Missing-file case tested.
- Tested without elevated privileges.
- Tested on the target OS.
- Logs reviewed.

12.4 Security

- Source is trusted and reviewed.
- Least privilege is used.
- Credentials protected; secrets not logged.
- External connections understood.
- Destructive actions controlled; backups exist.

13. Closing

A good script is not merely code that works. It is understandable, safe, tested, documented, repeatable, and built for the environment where it will run. For GreyNOC operations, review every script for purpose, permissions, security impact, reliability, and maintainability before it touches a real system.

TIP. Build scripts that are useful, safe, repeatable, and trusted. Everything in this guide serves that one goal.

GreyNOC · greynoc.com · Certified Penetration Testing & Ethical Hacking · Michigan
GN-OPS-SCRIPT-001 · v1.0 · Classification: PUBLIC · Published 2026-06-14